



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

PROJECT REPORT

STREAMIFY – REAL-TIME LANGUAGE EXCHANGE & COMMUNICATION PLATFORM

Submitted in partial fulfilment for the FullStack Project

Submitted by

Adarsh Agrawal 230911264

Department of Information Technology
MIT Manipal

1. Abstract

Streamify is a full-stack web application designed as a language learning and cultural exchange platform where users can meet people across the world, build connections, and communicate in real time. The core problem the project addresses is that most learners can study vocabulary or grammar online, but they often struggle to find actual speaking partners. Streamify attempts to solve that by creating a communication-first environment where users can register, build a language profile, discover suitable learners, send connection requests, chat, and even join calls within the same ecosystem.

The application was implemented using a modern MERN-style stack with React and Vite on the frontend, Node.js and Express on the backend, MongoDB for persistence, and Stream APIs for messaging and video-call functionality. Additional libraries such as Tailwind CSS, DaisyUI, React Query, Zustand, Axios, JWT authentication, bcrypt password hashing, and cookie-based session handling were used to make the system more production-like.

Unlike a generic demo that only shows static pages, Streamify follows an actual end-to-end product flow: signup, login, onboarding, recommendation of users, social connection handling, notifications, chat interface, and call support. The final system demonstrates not only frontend and backend implementation, but also practical product engineering decisions such as protected routes, user filtering logic, secure authentication, and third-party real-time service integration.

2. Introduction

Language learning has shifted from textbook-based memorization to interaction-driven practice. While many apps help users learn words or grammar, very few create a smooth path from learning to real conversation. Streamify was built around this missing layer: human interaction.

The project provides a platform where users can create a profile based on their native language and target language, discover relevant people, and communicate in real time. This makes the system more engaging and more useful than a one-way content platform. From a development perspective, the project was also chosen to explore full-stack web engineering in a realistic product setting.

2.1 Need for the System

Many language learners face a practical issue: they know what they want to learn, but they do not know whom to practice with. Existing tools are often fragmented—one app for vocabulary, another for speaking partners, and a separate service for calls or chat. This creates friction and lowers consistency.

Streamify brings these pieces together into a single platform where profile creation, partner discovery, connection management, messaging, and calling are integrated. This improves usability and makes language practice feel more natural and social.

2.2 Objectives of the Project

- Build a full-stack language exchange platform with a complete user journey.
- Implement secure authentication and protected access for users.
- Allow profile onboarding with native and learning language preferences.
- Create a friend request / connection system between users.
- Provide a chat and video-call experience using real-time communication services.
- Design a clean, responsive, dark-themed interface with multiple screens.
- Demonstrate practical use of frontend state management, API integration, and backend modularisation.

2.3 Scope of the System

For Users:

- Account creation and login
- Profile onboarding and avatar generation
- Language preference setup
- Friend discovery and requests
- Notifications for new connections
- Real-time chat and call support
- Learning resources section

For System/Admin Perspective:

- Authentication validation
- Secure user data handling
- Persistent user and request storage
- Scalable architecture for future feature additions

3. System Requirements

3.1 Hardware Requirements:

- Processor: Any modern CPU
- RAM: 4 GB minimum
- Disk Space: 1 GB minimum
- Device: Laptop/desktop with internet access

3.2 Software Requirements:

- Operating System: Windows / macOS / Linux
- Browser: Chrome, Firefox, Edge
- Frontend: React, Vite, Tailwind CSS, DaisyUI
- Backend: Node.js, Express.js
- Database: MongoDB Atlas / local MongoDB
- IDE: Visual Studio Code
- API Tools: Stream Chat and Stream Video

4. Technologies Used

Component	Technology
Frontend	React 19, Vite, Tailwind CSS, DaisyUI
Routing	React Router
State / Data Fetching	Zustand, TanStack React Query, Axios
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
Authentication	JWT, bcryptjs, cookie-parser
Real-Time Communication	Stream Chat, Stream Video SDK
Deployment/Env	dotenv, CORS, Nodemon, ESLint

5. System Design / Architecture

The project follows a client–server architecture. The React frontend handles UI rendering, routing, and user interaction. The Express backend exposes REST APIs for authentication, user discovery, friend request handling, and Stream token generation. MongoDB stores user records and request relationships.

Basic interaction flow:

- User performs an action on the React UI.
- Frontend sends API request to the Express backend using Axios.
- Backend validates request, applies business logic, and interacts with MongoDB.
- If communication features are required, a Stream token is generated or validated.
- Response is returned to the frontend and rendered in the UI.

Important design choices:

- JWT stored in HTTP-only cookies for safer session handling.
- Controllers and routes separated for cleaner backend structure.
- Onboarding status used to filter incomplete profiles from discovery.
- Stream identity synchronized with user onboarding/profile details.

6. Implementation

The project was implemented as a multi-page application with clear user flow. The following major modules were developed:

- Login & Registration: Users can create an account and securely sign in. Passwords are hashed before storage, and authenticated sessions are managed using JWT cookies.
- Onboarding: After signup, the user completes profile details such as full name, bio, native language, learning language, location, and avatar. This ensures that user discovery is meaningful rather than random.
- Home / Discovery: The home page displays current friends and recommended learners. Recommendations exclude the current user, existing friends, and incomplete profiles.
- Friend Request System: Users can send requests, receive requests, and build a friend network. Duplicate and self-requests are prevented in backend logic.
- Notifications: The notifications module displays social updates such as accepted requests and new connections.
- Resources Page: A lightweight learning support page provides language learning tips and useful external resources.
- About Us: An informational module describing the purpose of the platform and the technologies behind it.
- Chat Module: Once connected, users can open a chat channel and exchange messages in a clean messaging interface.
- Video Call Module: Users can initiate a call using Stream-powered video features, demonstrating real-time communication capability.

7. Results and Discussion

The final system successfully delivers the intended product flow. Users are able to register, log in, complete their language-learning profile, discover new learners, connect with other users, receive notifications, and start chats or calls. The UI maintains a consistent dark theme and presents the product as a polished communication platform rather than a disconnected set of forms.

A strong outcome of the project is that it combines both social and technical logic. For example, user recommendation is not just a list-all-users implementation; it intentionally filters incomplete or already-connected users. Similarly, communication was not simulated using dummy pages—actual chat and call workflows were integrated.

Streamify demonstrates:

- Frontend engineering and reusable component design
- Backend API design and controller-based organization
- Database modeling for users and social requests
- Secure authentication practices
- Real-time service integration
- End-to-end product thinking

8. Output Screens / Screenshots

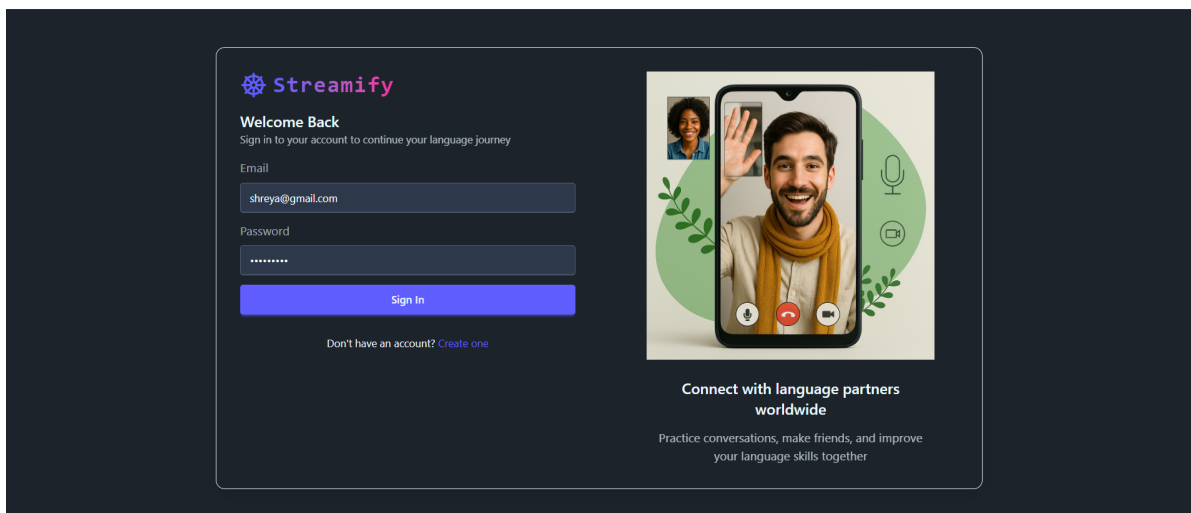


Figure: Login Page – secure sign-in interface for existing users.

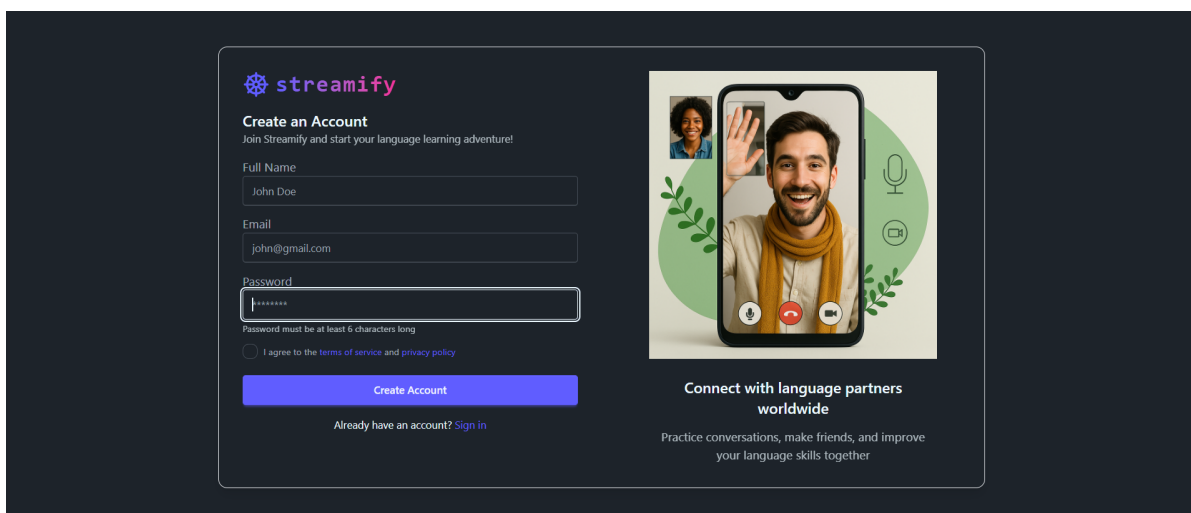


Figure: Registration Page – new user account creation workflow.

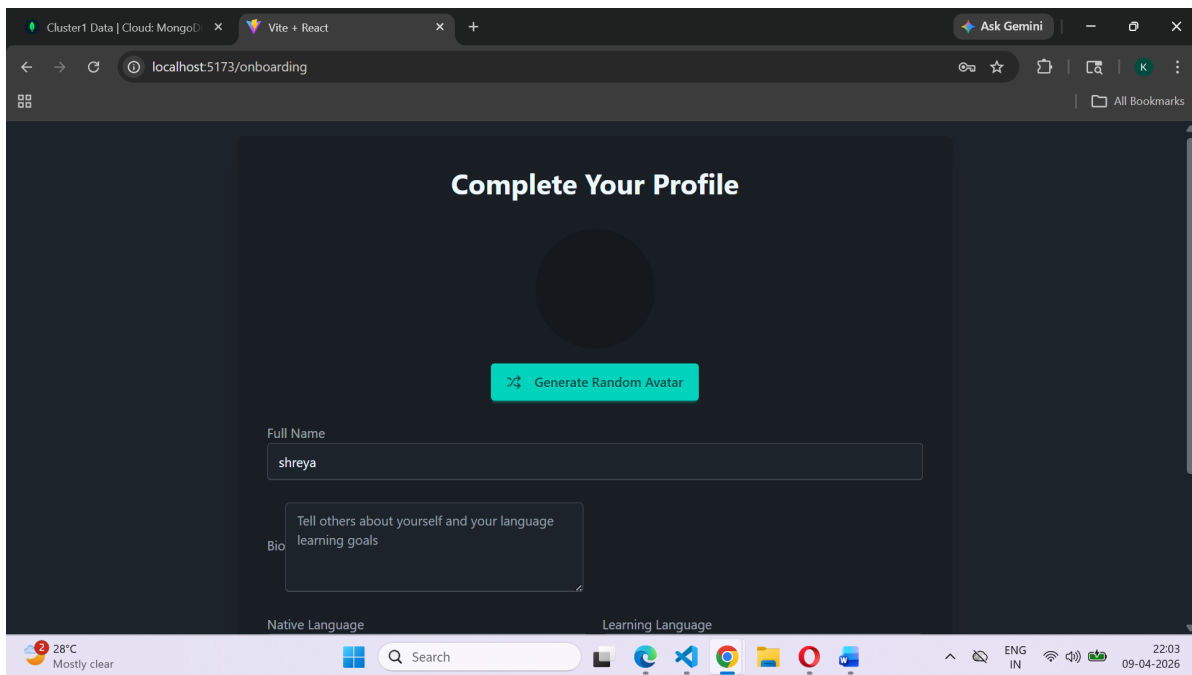


Figure: Onboarding Page (upper section) – profile completion and avatar generation.

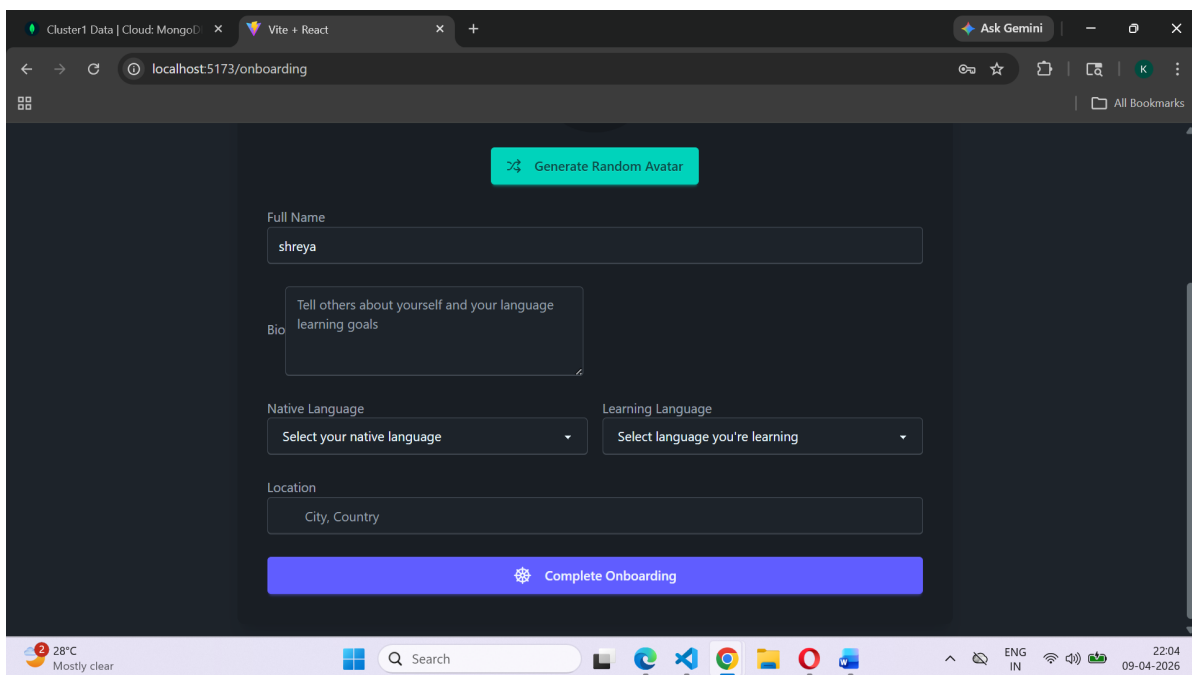


Figure: Onboarding Page (bottom section) – language preferences and location setup.

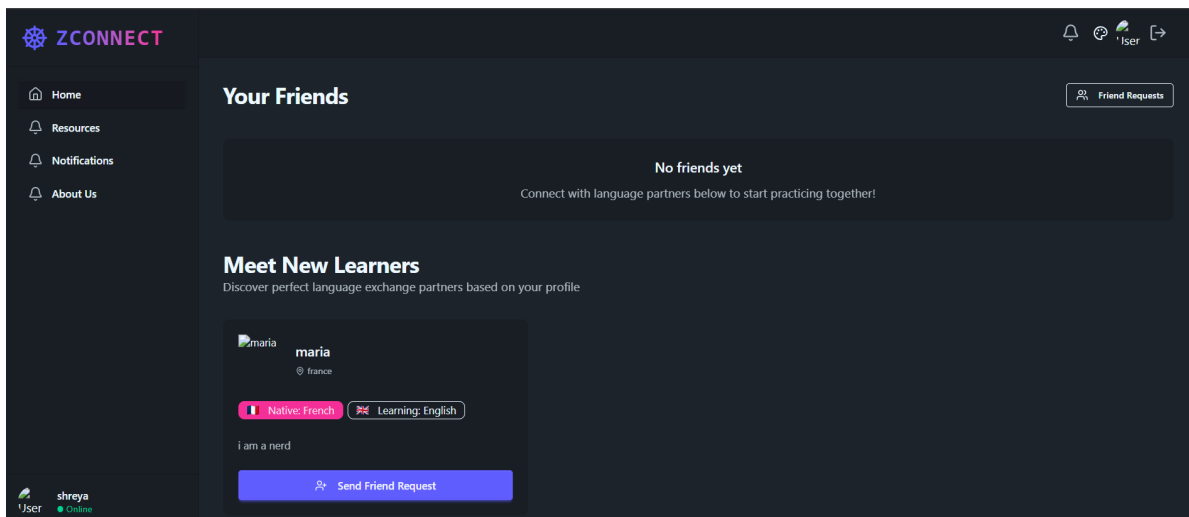


Figure: Home Page – friends section and learner discovery cards.



Figure: Resources Page – learning tips and curated resources.

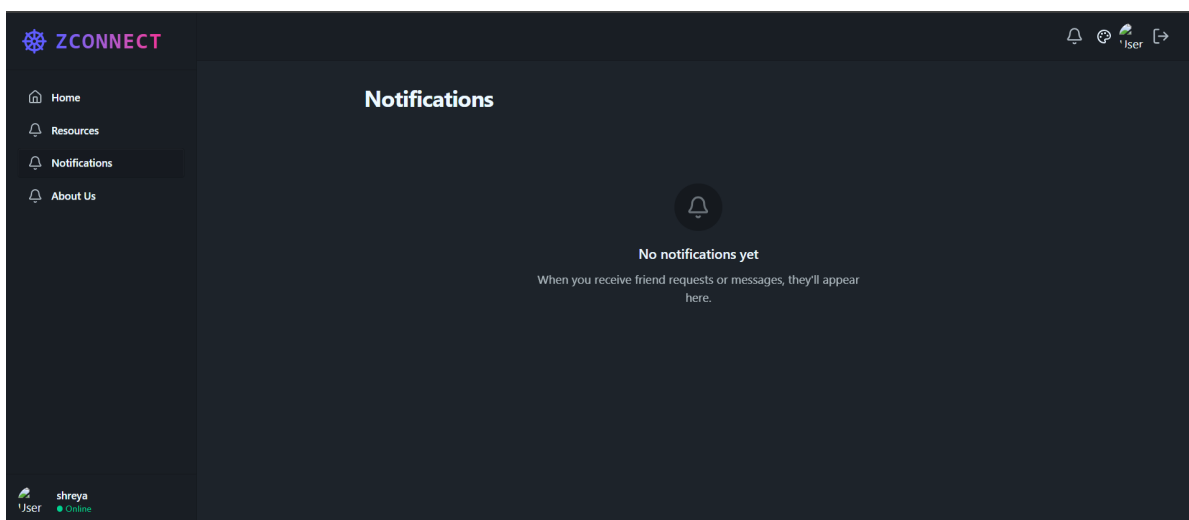


Figure: Notifications Page (empty state) – notification placeholder before connections.

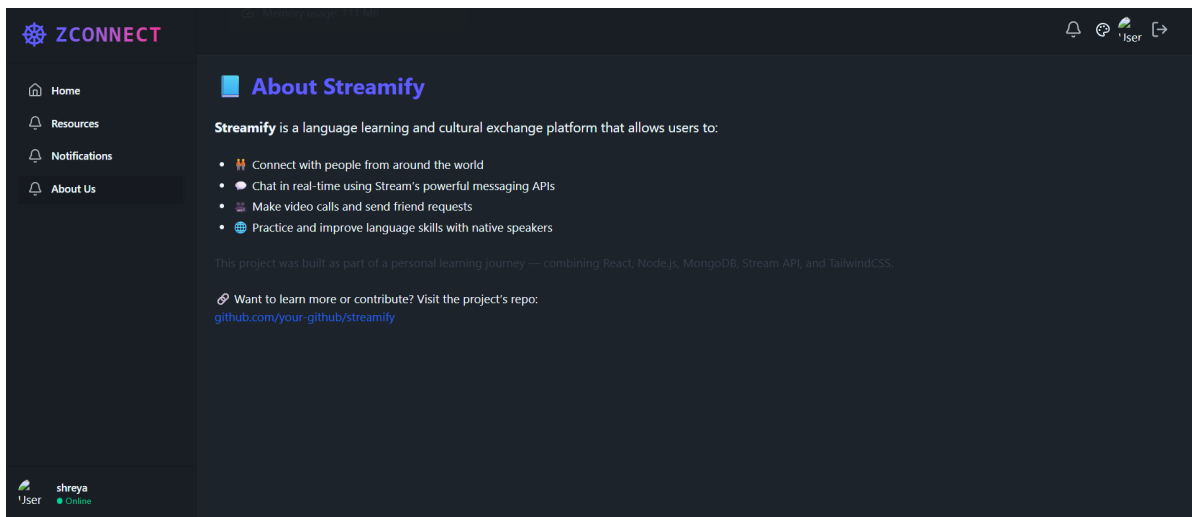


Figure: About Us Page – project description and purpose.

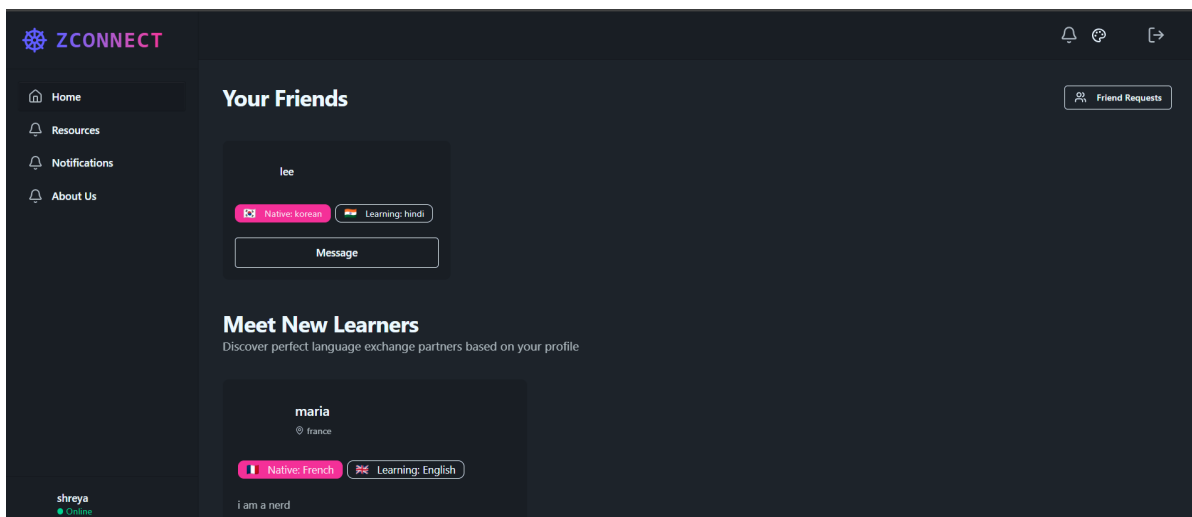


Figure: Home Page (with friend connected) – existing friend and recommendation visible.

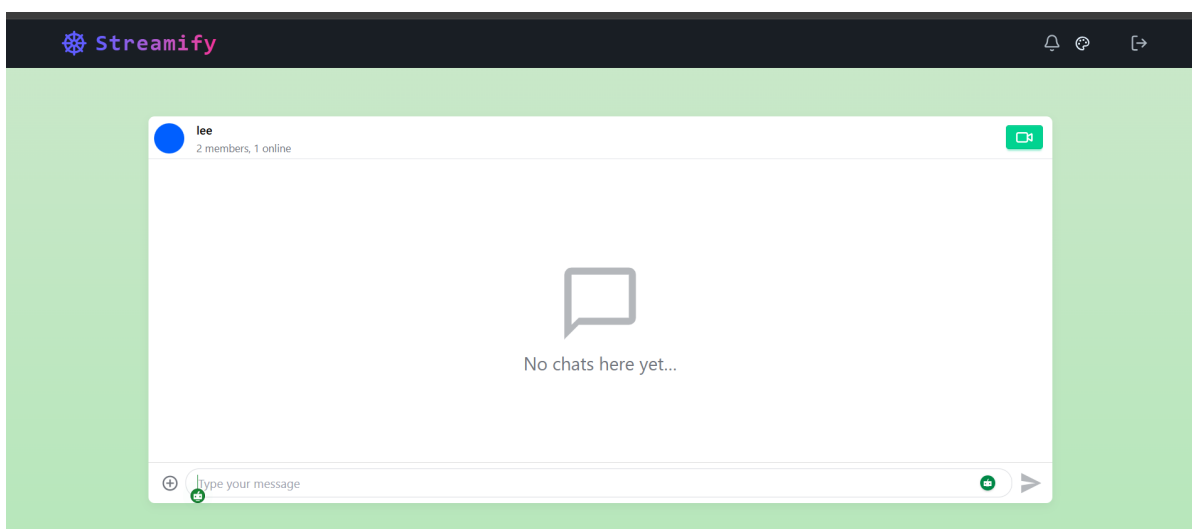


Figure: Chat Page – one-to-one conversation interface.

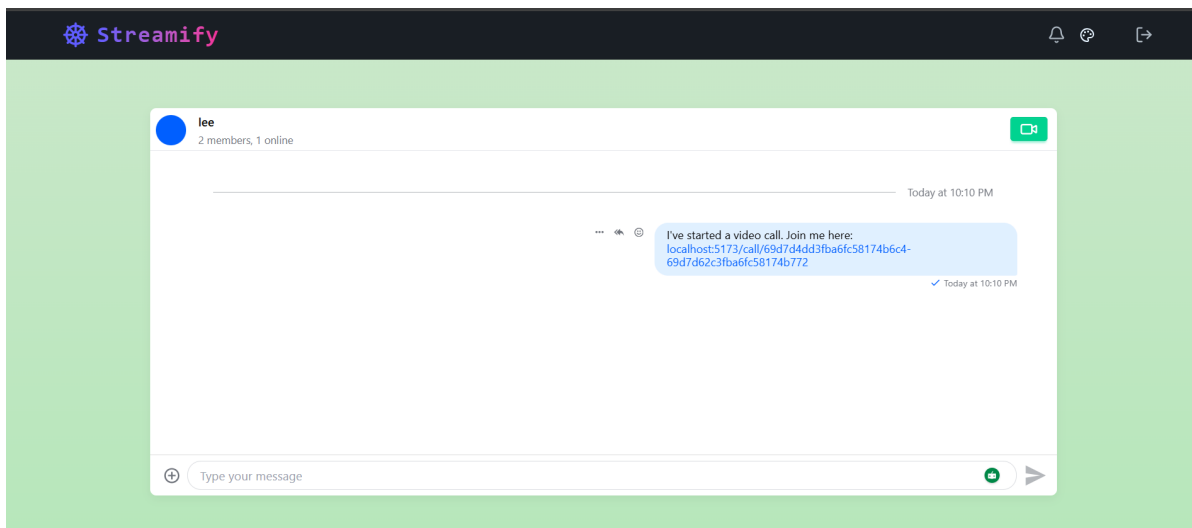


Figure: Chat Page with call invite message – message exchange and call link.

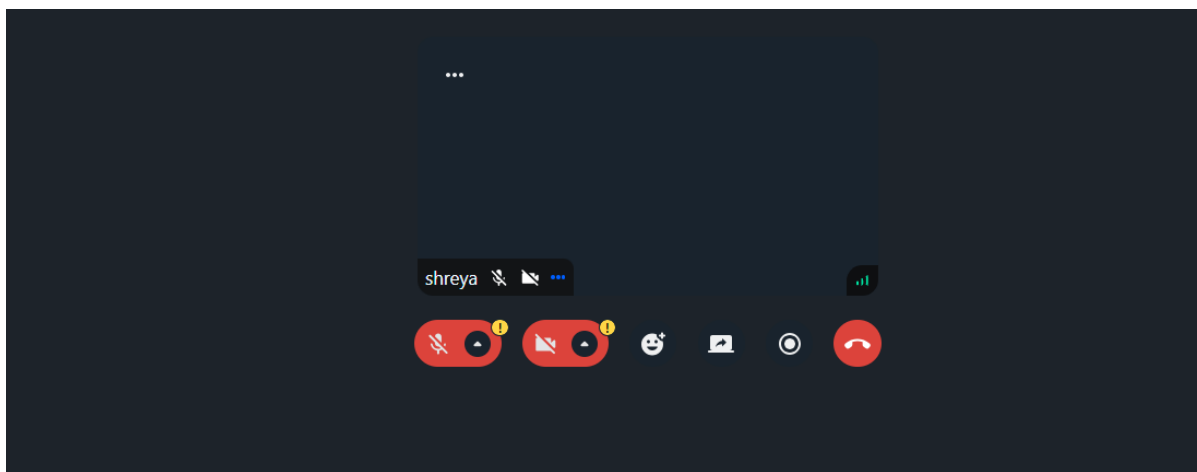


Figure: Video Call Interface – live call screen powered by Stream.

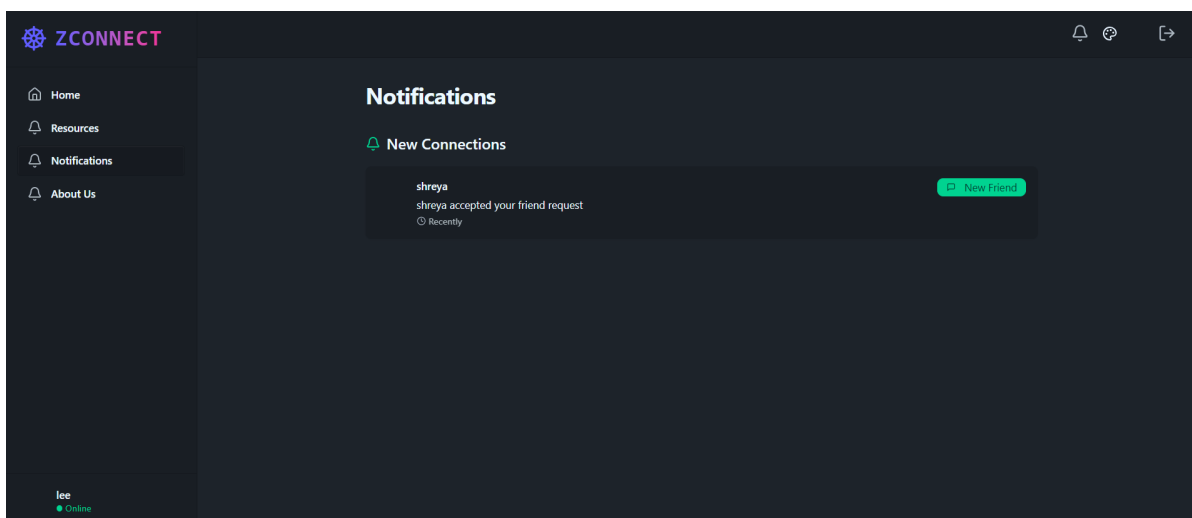


Figure: Notifications Page (active state) – accepted connection shown to user.

9. Feature-wise Result Summary

Feature	Result/Output
Secure Login System	Validated user login with hashed passwords and JWT-based authentication.
Profile Onboarding	Users can set bio, languages, location, and avatar before using the platform.
Learner Discovery	System shows suitable users instead of blindly listing every account.
Friend Request Workflow	Requests can be sent and accepted without duplication or self-request errors.
Notifications	Users receive social updates like new connections and request status.
Chat Integration	Connected users can open messaging channels and exchange messages.
Video Calling	Users can initiate and join video calls through Stream integration.
Responsive UI	Application remains clean and usable across major screens and flows.

10. Conclusion and Future Work

10.1 Conclusion

Streamify was successfully designed and implemented as a real-time language exchange platform that combines social discovery, secure authentication, messaging, and calling into a single web application. The project goes beyond static frontend design by supporting actual backend workflows, persistent data storage, and third-party communication integration. It demonstrates practical understanding of modern web development and reflects how a student project can be shaped into a product-oriented solution.

10.2 Future Work

Smart matching based on interests, language level, and activity.

Typing indicators, online presence, and unread message counters.

Group rooms and topic-based communities.

AI-generated conversation starters for beginners.

Calendar-based language practice scheduling.

Audio notes, media sharing, and translation assist features.

Deployment with production hosting and custom domain.

11. References

[1] React Documentation – <https://react.dev/>

[2] Vite Documentation – <https://vite.dev/>

[3] Node.js Documentation – <https://nodejs.org/>

[4] Express.js Documentation – <https://expressjs.com/>

[5] MongoDB Documentation – <https://www.mongodb.com/docs/>

[6] Mongoose Documentation – <https://mongoosejs.com/docs/>

[7] Stream Chat Documentation – <https://getstream.io/chat/docs/>

[8] Stream Video Documentation – <https://getstream.io/video/docs/>

[9] JWT Introduction – <https://jwt.io/introduction>

[10] Tailwind CSS Documentation – <https://tailwindcss.com/docs>